SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1518

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Dave's Taxonomy: Precision (Level 3) **Total Marks: 30**
Question Count: 3

Code of Conduct

I G. Jaswanth - 192472235 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Jaswanth

Assessment Tasks

Case Study

1. **Case Study 1: Smart Retail Inventory Management System**

   A retail chain implements a cloud-based inventory system connected to store devices.Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.
2. **Case Study 2: Cloud-Based Document Collaboration System**

   An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices.
   Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.
3. **Case Study 3: Online Food Ordering Platform**

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients.
Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform.
Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure

uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

CASE STUDY 1: SMART RETAIL INVENTORY MANAGEMENT SYSTEM

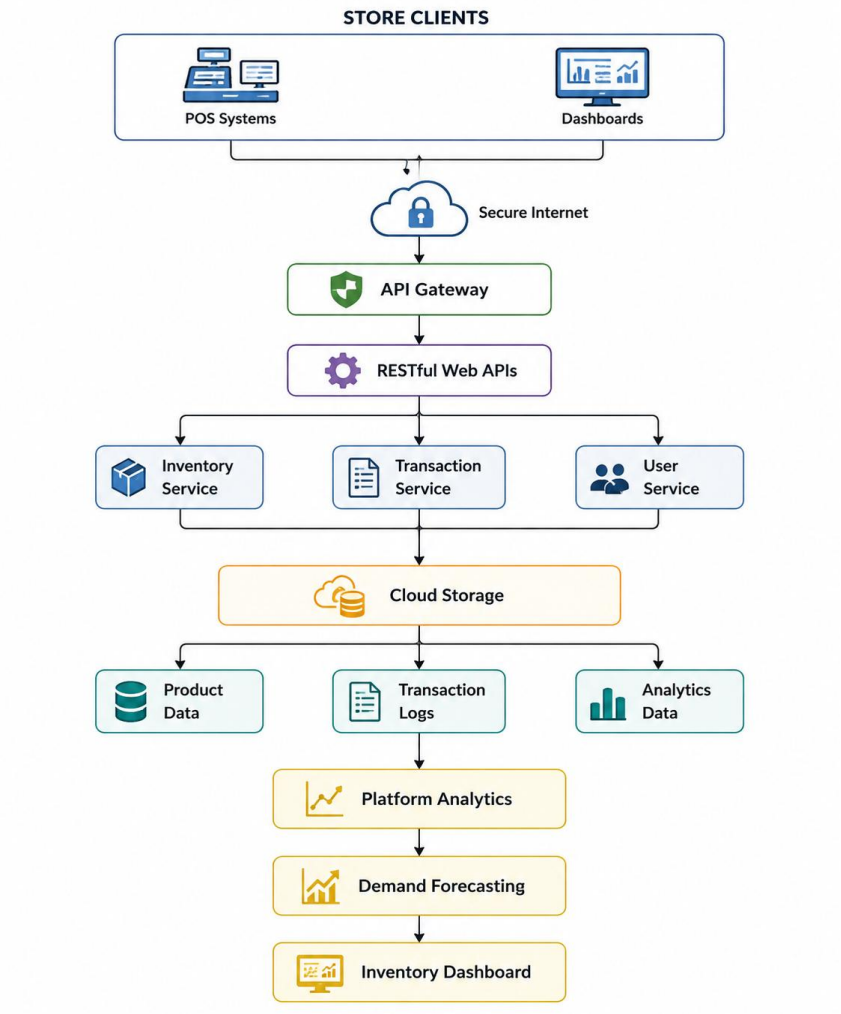
1. Understanding of the Case

The retail chain requires a centralized cloud-based inventory management system that connects **POS systems, store devices, and dashboards**. The system must allow clients to communicate with cloud services through **Web APIs**, store product and transaction information using **cloud storage**, and use **platform services for analytics and demand forecasting**.

The major requirements are:

- Centralized product and inventory management.
- Communication between POS systems and cloud services.
- Web API-based data exchange.
- Cloud storage for product data and transaction logs.
- Analytics and demand forecasting.
- Secure access through VPNs, firewalls, and API gateways.
- Scalable infrastructure for multiple stores.

2. Proposed Cloud Architecture



Security layer: VPN + Firewall + API Gateway + Authentication/Authorization

3. Cloud Concepts and Tools Applied

Web APIs

Web APIs provide communication between POS systems, dashboards, and centralized cloud services.

For example:

POS → REST API → Inventory Service

A POS system can send inventory updates whenever a product is sold.

Example:

POST /api/inventory/update

The API can update the corresponding product quantity in the cloud.

Cloud Storage

Cloud storage is used to store:

- Product information
- Inventory records
- Transaction logs
- Analytics datasets

It provides scalable storage without requiring every store to maintain its own storage infrastructure.

Platform Services

Cloud platform services process the collected data and generate useful insights.

They can be used for:

- Sales analysis
- Inventory analysis
- Demand forecasting
- Identifying frequently sold products
- Predicting future stock requirements

API Gateway

The API Gateway acts as the controlled entry point for API requests.

It provides:

- Request routing
- Authentication
- Authorization

- Rate limiting
- API monitoring

This prevents clients from directly accessing internal cloud services.

Security

The system uses:

- **VPN** for secure communication between stores and cloud infrastructure.
- **Firewalls** to control network traffic.
- **API Gateway** to secure API access.
- **Authentication and authorization** to restrict access to resources.
- **Encryption** to protect sensitive data during transmission and storage.

4. Working of the System

1. A customer purchases a product at a store.
2. The POS system records the transaction.
3. The POS sends the transaction through a secure Web API.
4. The API Gateway receives and validates the request.
5. The Inventory Service updates the product quantity.
6. Transaction information is stored in cloud storage.
7. Analytics services process accumulated sales data.
8. Demand forecasting predicts future product requirements.
9. Store managers view inventory and forecasting information through dashboards.
10. If stock levels become low, the system can help management initiate replenishment.

5. Analysis and Trade-offs

Requirement	Cloud Solution	Advantage
Multiple stores	Centralized cloud services	Easy management
POS communication	RESTful Web APIs	Standard communication
Large data volume	Cloud storage	Scalable storage
Sales analysis	Platform analytics	Faster insights
Demand prediction	Forecasting service	Better inventory planning
Secure access	VPN + Firewall + API Gateway	Improved security
Increasing stores	Cloud scalability	Easy expansion

Analysis

A centralized cloud architecture is more suitable than maintaining separate inventory systems at each store because all stores can access a common source of inventory information.

However, the system becomes dependent on network connectivity and cloud services. Therefore, secure networking, API monitoring, backup mechanisms, and availability measures should be implemented.

6. Proposed Solution

The recommended solution is a **cloud-based centralized inventory platform** consisting of:

- POS and dashboard clients
- API Gateway
- RESTful Web APIs
- Inventory and transaction services
- Cloud storage
- Platform analytics
- Demand forecasting
- VPN and firewall protection

This design allows the retail chain to add new stores without redesigning the entire infrastructure.

7. Benefits

- Centralized inventory management.
- Real-time communication between stores and cloud services.
- Scalable cloud storage.
- Improved demand forecasting.
- Reduced manual inventory management.
- Better visibility of sales and stock levels.
- Secure API-based communication.
- Easy expansion to additional stores.

8. Conclusion

The proposed cloud-based Smart Retail Inventory Management System integrates **Web APIs, cloud storage, and platform services** to provide centralized and scalable inventory management. POS systems and dashboards communicate with cloud services through secure APIs, while product data and transaction logs are stored in cloud storage. Platform analytics and demand forecasting help the retail chain make better inventory decisions. VPNs, firewalls, and API gateways provide secure access to the system. Therefore, the solution is **practical, scalable, secure, and suitable for real-world retail operations**.

CASE STUDY 2: CLOUD-BASED DOCUMENT COLLABORATION SYSTEM

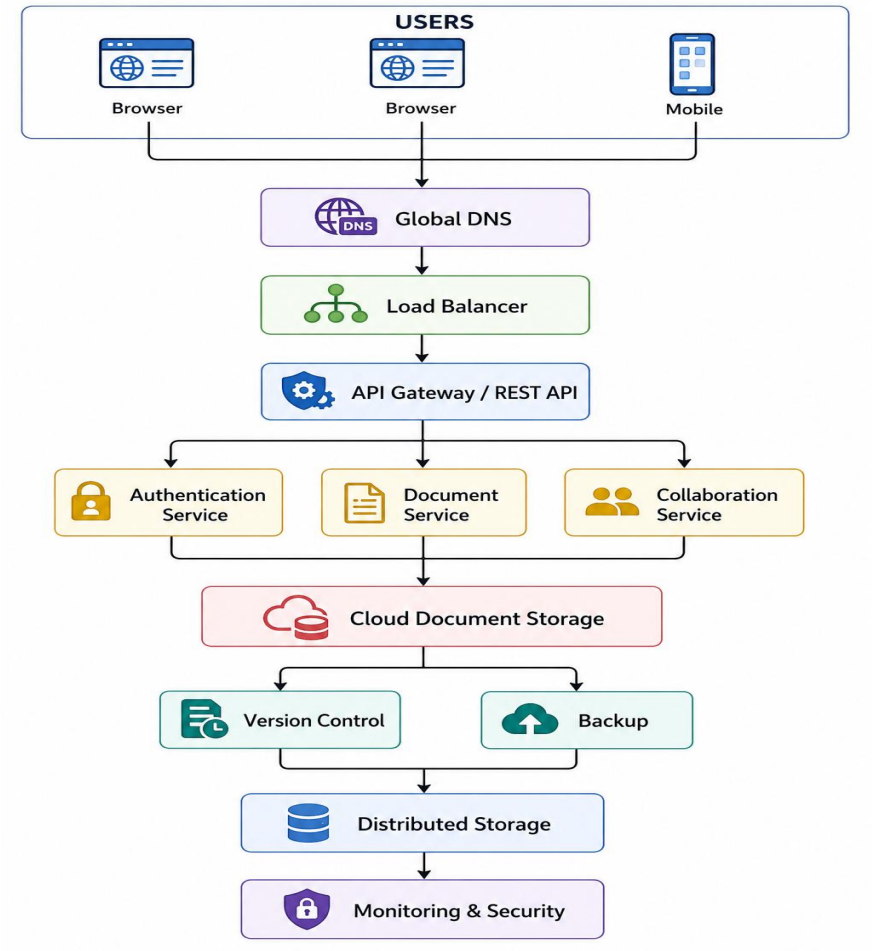
1. Understanding of the Case

The enterprise requires a cloud-based document collaboration system similar to Google Docs. Users should be able to access documents through browsers, edit them in real time, and share them with other users.

The major requirements are:

- Browser-based access.
- Real-time document editing.
- Document sharing and collaboration.
- Distributed cloud storage.
- Version control.
- Web APIs for synchronization.
- Support for multiple devices.
- Low-latency global access.
- High availability.
- Encryption and identity management.
- Access permissions for shared documents.

2. Proposed Cloud Architecture



3. Cloud Concepts and Tools Applied

Web APIs

Web APIs allow different clients to communicate with the cloud platform.

For example:

Browser → REST API → Document Service

When a user edits a document, the client can send the changes to the cloud through APIs.

APIs also allow multiple devices to synchronize the latest document version.

Cloud Storage

Documents are stored in distributed cloud storage.

The storage can contain:

- Text documents
- Images
- Attachments
- Document metadata
- Previous document versions

Distributed storage improves availability and allows users to access documents from different locations.

Version Control

Version control maintains previous versions of documents.

For example:

Document v1
↓
Document v2
↓
Document v3
↓
Current Version

If a user accidentally deletes or modifies important content, an earlier version can be restored.

Identity Management

Identity management controls who can access the documents.

Example roles:

User	Permission

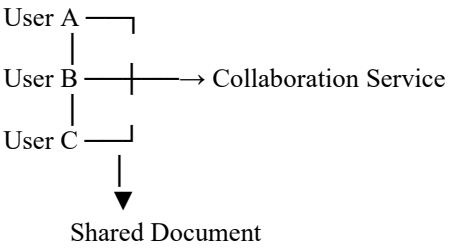
User	Permission
Owner	Read + Write + Share
Editor	Read + Write
Viewer	Read Only
Administrator	Manage Users

This prevents unauthorized users from modifying confidential documents.

4. Real-Time Collaboration

The system allows multiple users to work on the same document.

Example:



When one user makes an edit, the change is synchronized through the cloud so that other authorized users can see the updated content.

5. Working of the System

1. The user opens the document collaboration application through a browser.
2. The user authenticates using the identity management service.
3. The system verifies the user's permissions.
4. The client requests the document through a Web API.
5. The Document Service retrieves the document from cloud storage.
6. The user edits the document.
7. Changes are sent to the collaboration service.
8. The updated document is synchronized with other connected users.
9. A new document version is stored.
10. The document is continuously protected using encryption and access controls.

6. Security

The system uses multiple security mechanisms:

Encryption

Encryption protects documents while they are transferred and stored.

User → HTTPS/Encryption → Cloud

Identity Management

Users must authenticate before accessing documents.

Access Permissions

Document owners can specify whether another user can:

- View
- Edit
- Share

API Security

APIs should validate authentication tokens and permissions before processing requests.

7. Availability and Performance

Since users can be located across different regions, the system uses:

- Global cloud infrastructure.
- Distributed storage.
- Load balancing.
- Content delivery mechanisms where appropriate.
- Multiple cloud regions.
- Monitoring and health checks.

These mechanisms help reduce latency and maintain availability even if one infrastructure component fails.

8. Analysis and Trade-offs

Requirement	**Cloud Solution**	**Benefit**
Real-time editing	Collaboration service	Multiple users can work together
Document storage	Distributed cloud storage	High availability
Synchronization	Web APIs	Multi-device access
Version recovery	Version control	Prevents permanent data loss
Global access	Distributed infrastructure	Lower latency
Security	Encryption + IAM	Protects documents
High traffic	Load balancing	Better performance

Analysis

A cloud-based architecture is more suitable than a traditional file server because users can access and collaborate on documents from different locations and devices.

However, real-time collaboration requires continuous network connectivity and efficient synchronization. Large numbers of simultaneous users can also increase cloud resource consumption. Therefore, load balancing, scalable services, monitoring, and efficient synchronization mechanisms are required.

9. Proposed Solution

The recommended solution consists of:

- Browser/mobile clients
- Global DNS and load balancing
- API Gateway
- Authentication service
- Document service
- Collaboration service
- Distributed cloud storage
- Version control
- Backup
- Encryption
- Identity and access management
- Monitoring

This architecture provides a scalable platform for real-time document collaboration.

10. Benefits

- Real-time collaboration.
- Access from multiple devices.
- Centralized document management.
- Automatic version control.
- Easy document sharing.
- Improved availability.
- Scalable cloud storage.
- Secure access control.
- Reduced dependency on local storage.

11. Conclusion

The proposed Cloud-Based Document Collaboration System uses **cloud storage, Web APIs, and platform services** to provide real-time document editing and sharing. Distributed cloud storage ensures availability, while version control protects previous document versions. Web APIs synchronize data between multiple clients and devices. Encryption, identity management, and access permissions protect

documents from unauthorized access. Therefore, the proposed solution provides a **scalable, secure, highly available, and collaborative cloud platform**.

CASE STUDY 3: ONLINE FOOD ORDERING PLATFORM

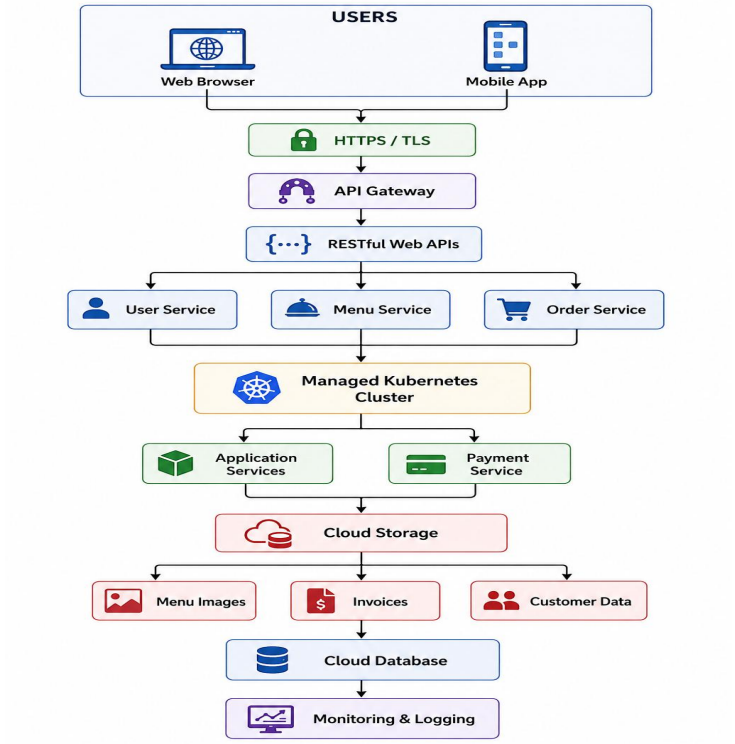
1. Understanding of the Case

The startup requires a cloud-based food ordering platform that can be accessed through **web browsers and mobile applications**. The frontend clients communicate with backend services using **RESTful Web APIs**. Cloud storage is used for menu images, invoices, and customer data. The system uses **virtual machines and managed Kubernetes** to provide scalability and reliability.

The major requirements are:

- Web and mobile client access.
- RESTful Web APIs.
- Cloud-based backend services.
- Cloud storage for menu images, invoices, and customer data.
- Virtual machines for application hosting.
- Kubernetes for container orchestration.
- Scalability and reliability.
- Authentication using tokens.
- HTTPS for secure communication.
- Role-Based Access Control (RBAC).

2. Proposed Cloud Architecture

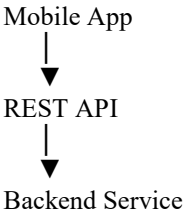


Security: Authentication Tokens + HTTPS + RBAC

3. Cloud Concepts and Tools Applied

RESTful Web APIs

REST APIs provide communication between the frontend and backend.



For example:

```
GET  /api/menu
POST /api/orders
GET  /api/orders/101
```

The APIs allow users to view menus, place orders, and track their orders.

Cloud Storage

Cloud storage is used to store:

- Restaurant/menu images
- Customer invoices
- Application files
- Other required data

This provides scalable storage and allows the application to access data from different locations.

Virtual Machines

Virtual machines provide computing resources for hosting application components.

They provide:

- Isolated computing environments.
- Flexible resource allocation.
- Reliable application hosting.

Kubernetes

Managed Kubernetes is used to deploy and manage containerized backend services.

It provides:

- Container orchestration.
- Automatic scaling.
- Load balancing.
- Service discovery.
- Self-healing.
- Rolling updates.

This helps the application handle increased order traffic.

4. Security Implementation

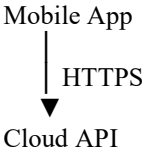
Authentication Tokens

Users receive an authentication token after successful login.

The token is validated before protected resources are accessed.

HTTPS

HTTPS encrypts communication between the client and cloud services.



This protects sensitive information during transmission.

Role-Based Access Control

RBAC provides different permissions to different users.

Role	Access
Customer	Browse menu, place orders, view own orders
Restaurant	Manage menu and orders
Delivery Staff	View assigned deliveries
Admin	Manage users, restaurants and system

5. Working of the System

1. The customer opens the food ordering application through a browser or mobile device.
2. The customer logs in using valid credentials.
3. The authentication service generates an access token.
4. The customer requests the available restaurant menu.
5. The request is sent through HTTPS to the REST API.
6. The backend retrieves menu information from the cloud database/storage.
7. The customer selects food items and places an order.
8. The Order Service processes the order through the Kubernetes cluster.
9. Order and invoice information are stored securely in the cloud.
10. Kubernetes automatically manages application instances based on workload.
11. Monitoring services track application performance and failures.

6. Scalability and Reliability

The application may experience high traffic during:

- Lunch hours
- Dinner hours
- Weekends
- Special offers
- Festivals

Managed Kubernetes can increase the number of application containers when demand increases.

Normal Traffic
↓
2 Containers

High Traffic
↓
5 Containers
↓
Kubernetes Auto Scaling

If a container fails, Kubernetes can restart it automatically, improving application reliability.

7. Analysis and Trade-offs

Requirement	Cloud Solution	Benefit
Web/Mobile access	REST APIs	Easy client integration
Secure communication	HTTPS	Protects data

Requirement	Cloud Solution	Benefit
User authentication	Tokens	Secure API access
Different user roles	RBAC	Controlled access
Menu images	Cloud Storage	Scalable storage
Application hosting	Virtual Machines	Flexible computing
High traffic	Kubernetes	Automatic scaling
Service management	Kubernetes	Easier deployment
Application reliability	Self-healing	Reduced downtime

Analysis

Using cloud infrastructure allows the food ordering platform to support a large number of customers without maintaining physical servers. REST APIs provide a standard way for web and mobile clients to communicate with backend services.

However, Kubernetes introduces additional management complexity, and cloud usage can increase costs as traffic grows. Therefore, resource limits, auto-scaling policies, monitoring, and proper security controls should be configured.

8. Proposed Solution

The recommended solution is a **cloud-native food ordering platform** consisting of:

- Web browser and mobile clients.
- HTTPS communication.
- API Gateway.
- RESTful Web APIs.
- Authentication service.
- User, Menu, Order and Payment services.
- Managed Kubernetes.
- Virtual machines.
- Cloud storage.
- Cloud database.
- Monitoring and logging.
- RBAC and authentication tokens.

This architecture provides scalability, reliability, security, and easy access for customers and restaurants.

9. Benefits

- Supports both web and mobile users.
- Secure API communication.

- Scalable application infrastructure.
- Efficient management using Kubernetes.
- Reliable cloud storage.
- Role-based access control.
- Automatic scaling during peak traffic.
- Reduced infrastructure management.
- Easy deployment of new services.

10. Conclusion

The proposed Online Food Ordering Platform effectively integrates **RESTful Web APIs, cloud storage, virtual machines, and managed Kubernetes** to provide a scalable cloud-based application. Authentication tokens, HTTPS, and RBAC protect users and application resources. Kubernetes provides automated deployment, scaling, and reliability for backend services, while cloud storage handles menu images, invoices, and customer data. Therefore, the proposed architecture provides a **secure, scalable, reliable, and practical cloud solution** for an online food ordering system.
